

Timetable Generator

Unit & Integration Testing Report

Backend Test Suite — pytest

Test run date	2026-09-23
Total test cases	40
Passed	40
Failed	0
Pass rate	100%
Total execution time	87.24 seconds

1. Introduction and Scope

This report documents the automated backend test suite for the Timetable Generator project and the results of its most recent run. The suite is split between **unit tests**, which check individual solver components in isolation (calendar math, diagnostics, weight resolution), and **integration tests**, which drive the full HTTP API end-to-end with FastAPI's TestClient (bootstrap an account, build a department, generate a timetable, validate it, publish it, and view it per user role). A third category — **property-based testing** — is used for the single most safety-critical claim in the system: that every solver-produced schedule is genuinely conflict-free.

The objective of this suite is not code coverage for its own sake, but confidence in the two claims that matter most for a scheduling product: (a) the solver never produces a timetable with a clash it was supposed to prevent, and (b) a set of real defects observed in earlier builds cannot silently return.

2. Testing Methodology and Tools

Tool / Technique	Role in this suite
pytest	Test runner and assertion framework for all unit and integration tests.
pytest-cov	Measures statement coverage of the app/ package during the run.
Hypothesis	Property-based testing — generates many random-but-valid department instances and checks a property holds for all of them, rather than checking one hand-picked example.
FastAPI's TestClient	Drives real HTTP requests against the actual FastAPI app in integration tests, with no network required.
Isolated SQLite per test	tests/conftest.py creates a fresh temporary database and rebuilds the schema for every test, so tests cannot leak state into one another.

3. Test Environment

Operating System	Windows 10 (build 26200)
Python version	Python 3.10.11
Test database	SQLite (temporary file per test session, via conftest.py)
Constraint solver	Google OR-Tools CP-SAT
Command	cd backend && pytest (coverage run: pytest --cov=app --cov-report=term-missing)

4. Results Summary by Module

Module	Type	Test cases	Passed	Failed
test_calendar.py	Unit	5	5	0
test_diagnostics.py	Unit	4	4	0
test_golden_regression.py	Unit (regression)	5	5	0
test_solver_smoke.py	Unit	2	2	0
test_validator_property.py	Property-based (Hypothesis)	1	1	0
test_weights.py	Unit	6	6	0
test_full_flow.py	Integration	5	5	0
test_phase4.py	Integration	8	8	0
test_rbac.py	Integration	4	4	0
Total		40	40	0

5. Detailed Test Cases

The tables below list every test case executed, grouped by module, with a plain-language description of what each one checks. All 40 cases passed on the run dated above.

5.1 Calendar / Contiguity Math (Unit)

Test case	What it checks	Result
test_slots_are_indexed_day_major_then_period	Bell-schedule slots are numbered day-first then period, so index arithmetic elsewhere is predictable.	PASS
test_contiguous_runs_break_at_a_gap	A run of contiguous periods correctly stops when a genuine timetable gap (e.g. no Period 5 on Friday) occurs.	PASS
test_duration_1_block_can_start_anywhere	A single-period class can be placed at any valid slot.	PASS
test_duration_2_block_cannot_start_where_no_contiguous_run_exists	A 2-period block (e.g. a short lab) is rejected from starting where the next period is not actually contiguous.	PASS
test_lunch_window_excludes_overlapping_starts	No class can be scheduled to start in a way that overlaps the lunch window.	PASS

5.2 Diagnostics (Unit)

Test case	What it checks	Result
test_no_time_slots_is_blocking	A department with zero bell-schedule slots is correctly flagged as a blocking pre-flight issue, not a silent failure.	PASS
test_demand_with_no_eligible_teacher_is_blocking_and_names_it	A subject with no qualified teacher is named explicitly in the diagnostic output.	PASS
test_sole_teacher_overload_is_named_with_arithmetic	When one teacher is the only option and is over-committed, the message states the actual hours short, not just 'infeasible'.	PASS
test_insufficient_lab_teachers_flagged	A lab needing more simultaneous teachers than are qualified is caught before solving starts.	PASS

5.3 Golden Regression Suite (Unit)

Each of these locks shut a specific defect that was observed in a previous build's real output — see Section 6.

Test case	What it checks	Result
test_subject_never_repeats_more_than_max_per_day	A subject cannot appear more times in one day than its configured daily maximum.	PASS
test_one_subject_never_gets_two_different_teachers_for_same_section	The same subject, for the same section, is never taught by two different teachers across the week.	PASS
test_lab_batches_prefer_running_in_parallel_not_serialised	Parallel lab batches are scheduled at the same time slot (parallel), not spread across different slots (serialised), when the objective allows it.	PASS
test_batch_mode_sequential_forces_batches_apart	When a lab is explicitly configured as 'sequential', its batches are kept apart in time as required.	PASS
test_batch_mode_merged_shares_one_room_and_teacher	When a lab is configured as 'merged', all its batches correctly share a single room and teacher.	PASS

5.4 Solver Smoke Tests (Unit)

Test case	What it checks	Result
test_small_instance_solves_and_validates_clean	A small, feasible department instance solves successfully and the independent validator reports zero violations.	PASS
test_infeasible_when_teacher_insufficient_for_parallel_labs	A deliberately under-resourced instance (too few teachers for parallel labs) is correctly reported infeasible rather than silently dropping a class.	PASS

5.5 Property-Based Validator Test (Hypothesis)

This is the highest-value test in the suite: instead of one example, Hypothesis generates many randomised-but-valid department instances and asserts the property holds for every one it tries.

Test case	What it checks	Result
test_feasible_schedules_are_always_valid	For any solver-feasible randomly generated department instance, the independently-written validator finds zero constraint violations in the resulting schedule.	PASS

5.6 Weight Resolution (Unit)

Test case	What it checks	Result
test_every_soft_rule_key_is_read_by_the_model_builder	Every soft-rule key the UI can present is actually consumed somewhere in the model builder (no orphaned settings).	PASS
test_every_preset_only_uses_real_keys	Built-in presets (Balanced / Student-friendly / Teacher-friendly) reference only rule keys that genuinely exist.	PASS
test_resolve_weights_fills_missing_keys_with_nice_to_have_default	An institution that hasn't configured a given rule gets a sensible default weight rather than a missing/zero one.	PASS

Test case	What it checks	Result
test_unknown_level_falls_back_to_nice_to_have	An unrecognised importance label degrades safely instead of raising an error or being silently ignored.	PASS
test_very_important_outweighs_several_nice_to_have	The relative ordering of importance levels actually produces the expected relative weighting in the objective.	PASS
test_is_must_have	Correctly identifies which importance level promotes a soft rule into a hard constraint.	PASS

5.7 Full Generation Flow (Integration)

Drives the real FastAPI app end-to-end via HTTP, exactly as the frontend would.

Test case	What it checks	Result
test_full_generation_flow	Bootstrap an admin, create a department, generate a timetable, validate it, publish it, and confirm a logged-in teacher/student sees only their own scoped view.	PASS
test_ai_timetable_assistant_apply_is_run_scoped_only	An AI-proposed edit only ever applies to the run it was generated for, never leaking into other runs.	PASS
test_merged_lab_batches_share_room_and_teacher_end_to_end	The 'merged lab batch' behaviour from Section 5.3 holds true through the full API, not just inside the solver.	PASS
test_infeasible_generation_reports_named_cause	A generation request that cannot succeed returns a specific, named cause via the API, not a generic error.	PASS
test_fit_into_existing_scope_freezes_other_sections	Generating for one department in 'fit_into_existing' mode cannot double-book a teacher shared with an already-published department.	PASS

5.8 Extended Feature Flow (Integration)

Test case	What it checks	Result
test_reset_scoping_keeps_timetable_history	Resetting a scope does not delete unrelated historical runs.	PASS
test_editing_a_class_creates_a_child_run_and_leaves_parent_untouched	A manual slot edit creates a new version instead of mutating the published run in place.	PASS
test_teacher_marked_unavailable_is_never_scheduled_then	A teacher's declared unavailability window is genuinely respected by the generated schedule.	PASS
test_teacher_daily_cap_is_respected	No teacher exceeds their configured maximum teaching hours in a single day.	PASS
test_teacher_weekly_cap_is_respected	No teacher exceeds their configured maximum teaching hours in a week (24-hour default cap).	PASS
test_teacher_continuous_cap_is_respected	No teacher is scheduled for more consecutive periods than their configured continuous-teaching limit.	PASS
test_hard_constraint_rule_changes_the_result	Toggling a rule from soft to hard actually changes the generated schedule, proving the rule is live rather than cosmetic.	PASS
test_export_merges_multi_period_block_and_renders_lunch_band	The Excel/PDF export correctly merges a multi-period block into one visual cell and renders the lunch band.	PASS

5.9 Role-Based Access Control (Integration)

Test case	What it checks	Result
test_hod_cannot_touch_another_departments_run	A Head of Department cannot view or modify a timetable run belonging to a different department.	PASS
test_hod_can_generate_and_publish_own_department	A HOD retains full generate/publish rights within their own department.	PASS
test_register_requires_department_for_hod	Registration rejects a HOD account that isn't tied to a department.	PASS
test_admin_only_can_deactivate_users	Only an admin account can deactivate a user; other roles are rejected.	PASS

6. Why the Golden Regression Suite Exists

Three of the tests in Section 5.3 do not test a rule in the abstract — they each pin down a specific defect that was actually observed in a previous build's generated output: a subject appearing three times in one day for the same section, one subject being taught by two different teachers for the same section in the same week, and lab batches being scheduled one after another instead of in parallel when parallel scheduling was clearly possible. Writing a regression test directly from a real observed defect, rather than only from the specification, is what keeps a fixed bug from quietly coming back in a later change.

7. Code Coverage

A coverage run over the full app/ package recorded **64% statement coverage** (3202 of 5016 statements executed). Coverage is concentrated in the modules that carry the actual scheduling logic and its safety checks:

Module	Coverage	Note
<code>solver/types.py</code>	100%	Core data structures, exercised by every test.
<code>solver/weights.py</code>	100%	Fully covered by the weight-resolution unit tests.
<code>solver/calendar.py</code>	95%	Contiguity/slot-indexing math.
<code>solver/diagnostics.py</code>	94%	Pre-flight blocking-issue detection.
<code>solver/model_builder.py</code>	77%	CP-SAT encoding; core constraint paths are covered, some rarely-hit edge branches
<code>solver/engine.py</code>	81%	Solve loop and alternative-generation logic.
<code>solver/validator.py</code>	75%	Independent post-hoc schedule checker.
<code>solver/data_loader.py</code>	65%	Many branches here handle rarer data shapes (e.g. specific import edge cases) n
<code>services/exporters.py</code>	55%	Export formatting has the least direct test coverage; verified manually via generat

Coverage is intentionally uneven by design, not oversight: modules that decide whether a schedule is correct (calendar, diagnostics, weights, the validator) are held to a much higher bar than presentation-layer code such as export formatting, where a visual defect is low-risk and easy to spot by eye.

8. Conclusion

All 40 automated test cases pass on the current codebase, with zero failures and zero errors. The suite covers the three things that matter most for a scheduling system: individual solver mechanics (unit tests), the system behaving correctly end-to-end as a real client would use it (integration tests), and a mathematically strong guarantee that any schedule the solver produces is actually valid (the property-based validator test). Combined with the golden regression suite, this gives reasonable confidence that the system is both correct today and resistant to previously-fixed defects resurfacing.